

MicroSplat: Towards Automating 3D Small-Scale Reconstruction

Roman Mineyev[†], Robert Azarcon[†], Suvrat Jain, Chuizheng Kong, and Shreyas Kousik

Abstract—Reconstructing sub-millimeter detail of small objects underpins tasks such as circuit-board defect detection, specimen digitization, and archiving. 3D Gaussian Splatting (3DGS) makes photorealistic reconstruction possible from posed images alone, but macro optics violate two conditions it depends on. The field of view (FOV) and the depth of field (DoF) shrink to millimeters, so viewpoint error can take a sample out of frame, and only a thin focal plane is sharp in frame. Existing automated macro-capture rigs handle the shallow DoF with per-view focus stacking, but typically rely on classical photogrammetry and offline, human-in-the-loop processing. We present MicroSplat, which generates a 3DGS using a low-cost consumer microscope on a manipulator arm. After a single up-front object-localization step, the system runs unattended. It performs calibration, plans collision-free viewpoints, captures and fuses a focal stack at each viewpoint, segments the object across all views, and optimizes a 3DGS model warm-started with the arm’s own poses. We further compare against blur-aware 3DGS methods, which infer sharp scenes from defocused images, and find that measuring sharpness with a controlled focal sweep outperforms inferring it. We provide real and synthetic datasets (including robot and camera parameters) and the fitted 3DGS reconstructions.

I. INTRODUCTION

Centimeter-scale objects resolved at sub-millimeter detail are the working domain of a range of inspection and digitization tasks: defect detection on printed circuit boards (PCBs) and solder joints, the digitization of insects and biological specimens, and the archiving of coins and other small artifacts [1–4]. 3D Gaussian Splatting (3DGS) [5] makes photorealistic reconstruction possible from posed RGB images alone, but macro optics break two of its working assumptions at once. The field of view and the depth of field (DoF) both shrink to millimeters, so viewpoint error decides whether the sample is in frame at all; and within a single exposure only a thin focal plane is sharp, so recovering sharp detail requires an explicit strategy. Two families exist: focus stacking, which recovers sharpness before reconstruction by combining multiple images per viewpoint, and blur-aware methods, which recover a sharp scene from defocused images during the reconstruction phase. Systems that automate macro capture, such as the Orbitoscope [6], already handle the shallow DoF with per-view focus stacking, positioning their sample by hand at a fixed orbit center. What they do not produce is a Gaussian Splat; these systems reconstruct with classical photogrammetry and rely on offline, human-in-the-loop processing. The manual efforts that do create a Gaussian Splat require labor-intensive hand-masking of hun-

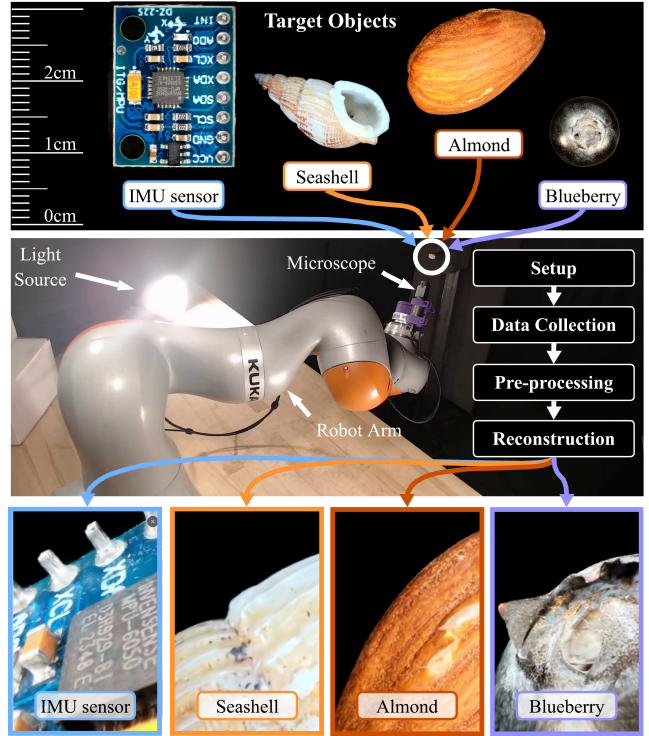


Fig. 1. Overview of our proposed MicroSplat pipeline for automatically generating photorealistic reconstructions of centimeter-scale objects at sub-millimeter detail.

dreds of images to suppress floater artifacts (spurious semi-transparent Gaussians) induced by blurred backgrounds [7].

MicroSplat¹ pairs a robotic manipulator arm (KUKA iiwa14 R820) with a low-cost consumer microscope as its end-effector (Fig. 1). We leverage the arm’s ± 0.1 mm repeatability to enable microscope calibration and precise microscope placement for focus stacking. Our use of a low-cost microscope introduced noise in both our images and poses. We developed several engineering solutions such as a human-in-the-loop object-centering pipeline, SfM initialization with robot pose and camera intrinsics, and Gaussian-fit-based window selection for focus stacking to make our pipeline robust to this noise.

To summarize, we make three main contributions:

1. **An autonomous capture-to-reconstruction pipeline**, in which, after calibration and object-centering, the system captures data and generates a 3DGS model of a small object. We also extensively evaluate the impact of

[†]Equal contribution.

All authors are with the Georgia Institute of Technology. Corresponding author: Roman Mineyev, rmineyev3@gatech.edu.

¹Project page (includes interactive viewer): <https://microsplat.github.io/>. Code: <https://anonymous.4open.science/r/Microscopic-3D-Reconstruction-FA3D>

key engineering choices in our pipeline (object centering and robot pose SfM initialization) through ablations and comparison to baselines.

2. **A comparison against blur-aware 3DGS methods**, in which we benchmark our focus-stacking approach on synthetic data using ground-truth in-focus renderings, and show that when a controlled robotic focal sweep captures sharp detail from every viewpoint, measuring sharpness directly outperforms inferring it from defocused images during reconstruction.
3. **A reconstruction dataset** of eight real objects and eight synthetic objects, spanning a wide range of attributes. The dataset contains information on scans, 3DGS reconstructions, initial robot arm poses and camera intrinsics. For each synthetic object we additionally release the Blender scene and object mesh, along with both the shallow-DoF focal stacks and the corresponding fully in-focus renderings, which serve as ground truth.²

The rest of the paper is organized as follows: Section II covers related work, Section III details our method’s hardware, calibration, planning, and reconstruction stack, and Section IV presents experimental results.

II. RELATED WORK

We now review existing systems for automated 3D capture, highlighting the gap in small-scale capture that our system targets.

A. Automated Capture Rigs

3D reconstruction is an essential tool used across a wide range of domains such as film, video games, virtual and augmented reality, medical imaging, and robotics. Neural rendering has emerged as a powerful approach to this problem, with Neural Radiance Fields (NeRF) [8] and 3DGS [5] being commonly adopted over classical photogrammetry for their ability to produce photorealistic novel-view synthesis from posed images alone.

With so many applications relying on 3D reconstruction, much work has focused on automating the data collection process. For example, the Google Scanned Objects dataset

demonstrated a custom enclosure with structured light and machine vision cameras capable of generating high-fidelity 3D models for over 1,000 household items for the downstream task of robot learning [9].

More recently, automated pipelines have moved toward reconstructing objects directly with 3DGS from RGB images alone, without the depth or structured-light hardware these earlier systems require [10–13]. For example, OmniScan [10] used a bi-manual robot to rotate a grasped object in front of a fixed RGB camera, merging two scans into a 3DGS model. To isolate the object from the gripper and background, it prompts SAM 2 [14] with foreground cues and scores the resulting candidate masks. We similarly use SAM 2 for object mask generation in MicroSplat.

B. Small-Scale Reconstruction

Beyond standard objects, there is great interest in applying 3D reconstruction to resolve sub-millimeter detail as well. In electronics and semiconductor manufacturing, high-fidelity reconstructions of PCBs, microcontrollers, and solder joints enable automated defect detection and quality control [1, 2]. Similar techniques are used to digitize insects and biological specimens in entomology, to archive coins, engravings, and other small artifacts in cultural heritage, and to characterize surface morphology in materials science [3, 4, 15, 16].

C. Small-Scale Capture and Sharpness Recovery

Hobbyists such as Danny Bittel have shown that Gaussian Splatting of small objects is possible with manual macro photography [7], using focus stacking [17] to overcome shallow DOF: a series of images are captured from a single viewpoint while the camera steps along the optical axis, and the resulting slices are fused into one all-in-focus composite.

Manually producing such a capture, however, faces two bottlenecks. First, at macro working distances, both FOV and DOF are small, so precise viewpoint control is essential. Second, masks are necessary since blurred backgrounds lack geometric cues, causing background-colored floater artifacts that forced Bittel to hand-mask hundreds of images.

Several works do automate this capture process for small samples via focus stacking [6, 15, 18]. Closest to this work, Orbitoscope [6] uses a custom six-axis gantry with a dedicated stacking axis for per-view focal stacks. Its viewpoints are fixed and the sample is manually positioned at the orbit center with intersecting laser pointer. We take the complementary approach of leaving the sample untouched and adjusting the viewpoints (i.e., the robot’s motion) to the estimated sample center.

All of these autonomous systems for small-scale objects, however, reconstruct with classical photogrammetry rather than Gaussian Splatting. To our knowledge, MicroSplat is the first fully autonomous pipeline for small-scale objects that goes from robotic data collection to a 3DGS model without human intervention after setup.

An alternative to capture-time focus stacking is to recover sharpness during reconstruction: Deblurring 3DGS [19] and its successors [20, 21] jointly estimate a blur model and

²<https://huggingface.co/datasets/splatmicro/microsplat>, released under CC BY 4.0.

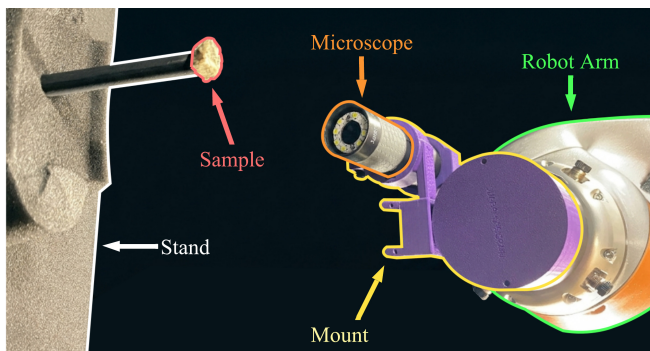


Fig. 2. Hardware setup. (Left) Sample mounted onto the stand and (right) KUKA iiwa14 R820 with custom microscope end-effector.

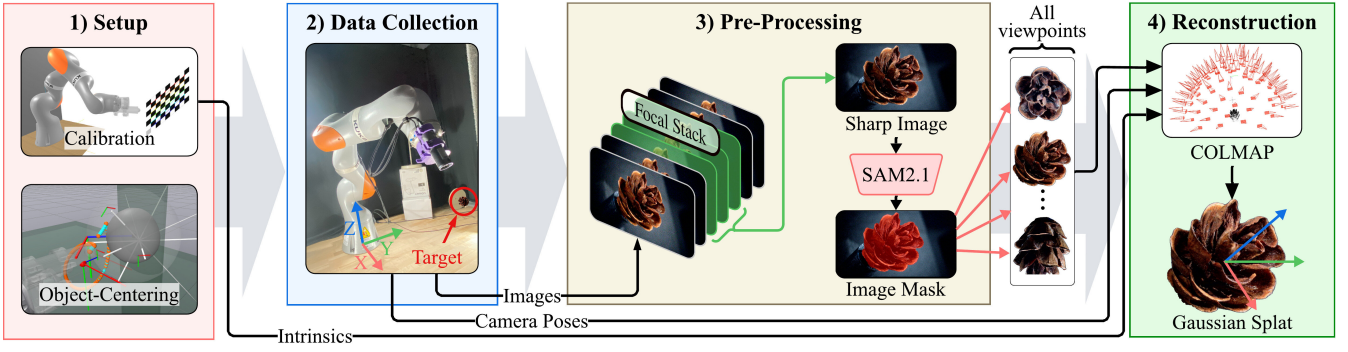


Fig. 3. Overview of our proposed method. Note the checkerboard in the calibration step is not drawn to scale.

a sharp scene from defocused images. However, with a robot arm we can capture in-focus detail directly from every viewpoint rather than inferring it. We compare these methods in Section IV-E.

III. PROPOSED METHOD

We address the problem of producing accurate, high-fidelity renderings of small-scale objects from images captured with a digital microscope. At this scale, the shallow DoF prevents any single image from being fully in focus (Fig. 4), and the capture precision required is impractical to achieve manually. We present a robotic pipeline that autonomously performs image acquisition and 3D Gaussian Splatting reconstruction. To proceed, we first describe our hardware, then our reconstruction pipeline, which has four steps (Fig. 3): setup, data collection, pre-processing, and reconstruction.

A. Hardware

1) *Robot*: We use a seven-degree-of-freedom KUKA iiwa14 R820 arm fitted with a custom end-effector that mounts an optical microscope (Fig. 2). The iiwa14 provides ± 0.1 mm pose repeatability, which is essential for keeping small samples within the microscope’s narrow field of view and shallow DoF across all viewpoints.

2) *Sample*: We place the sample on a fixed stand pointing towards the $-X$ axis of the robot’s base frame (see Fig. 3 for the robot’s coordinate frame) to maximize the spherical coverage of viewpoints, as checked with standard reachability analysis.

3) *Microscope*: We use a low-cost hobbyist microscope (35 USD on Amazon at time of writing) that uses a Galaxy-Core GC2063 1/2.9” RGB Bayer CMOS sensor with a resolution of 1920×1080 (2 MP) and a pixel pitch of $2.8 \mu\text{m}$. The microscope has a zoom knob, which we kept fixed (we leave changing the zoom level to future work). We used a calibration target of known dimensions (2 mm checkerboard squares with 9×6 interior corners) to calculate the object-space sampling of approximately $16.44 \mu\text{m}/\text{px}$ and FOV of 31.57×17.76 mm. The microscope’s camera image parameters, such as exposure and white balance, cannot be manually controlled. Because 3DGS requires photometric consistency across all views, we controlled the illumination

of our workspace instead, using four 800 lm, 5000 K lamps, evenly spread out behind the robot arm.

B. Setup

This first step is the pipeline’s only human-in-the-loop stage, comprising a calibration phase performed once and an object-centering phase repeated for every new sample.

1) *Calibration*: We used a 9×6 2mm-square inkjet-printed checkerboard pattern as the calibration target. Our use of a robot arm solves the difficult of microscope calibration: the high precision needed to keep a calibration target both in the FOV and fully in-focus. We generated viewpoints and planned the robot’s motion as detailed below in Section III-C, but without collecting focal stacks. Instead, at each viewpoint, we manually jogged the robot arm along any of the three camera-frame axes in 1 mm increments till the calibration target was in-focus as possible. The robot flange pose at each viewpoint was stored for hand-eye calibration. Intrinsic calibration yielded $f_x = 6488.95$, $f_y = 6196.35$, $c_x = 865.83$, $c_y = 565.52$ (all in px) with a re-projection error of 0.2161 px, and Daniilidis hand-eye calibration [22] yielded a rotation residual of 0.00901 rad and a translation residual of 1.0707 mm.

2) *Human-in-the-Loop Object-Centering*: We assume the sample stays static during data collection. However, without an accurate initial location of the sample, it may be off-center or even entirely outside the image frame during data collection.

We address this with a manual object centering step. We start with a rough guess of the sample location (either known from a previous sample, or by moving the robot manually). We then compute five poses to use for centering the object. The first pose aligns the camera’s optical axis with the robot base X-axis. The remaining four poses tilt the camera up, left, and right 60° , and downward 45° (to avoid collision with table), relative to the estimated object center.

The robot plans motion to each pose with RRT* (implemented in Drake [23]). Then, the user jogs the camera in 1 mm increments until the sample is centered on a crosshair (shown in the user’s camera feed), denoting (c_x, c_y) coordinates calculated from calibration. The 3D ray in camera

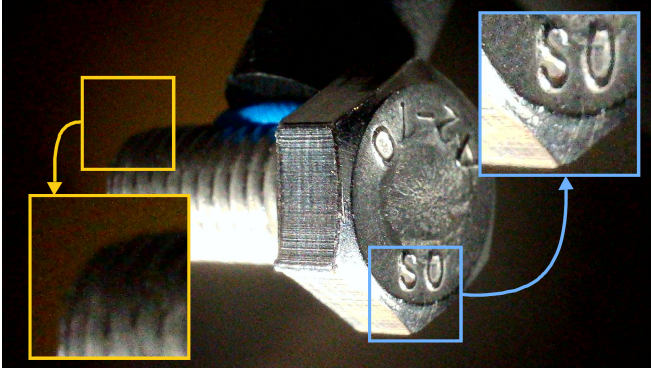


Fig. 4. A bolt imaged during our data collection. The letters are in focus, but the threads are blurry due to the microscope’s shallow depth of field.

coordinates passing through the pixel at (u, v) is

$$\mathbf{d} = \begin{bmatrix} (u - c_x)/f_x \\ (v - c_y)/f_y \\ 1 \end{bmatrix}. \quad (1)$$

By manually aligning the sample to (c_x, c_y) , this ray is simplified to $[0, 0, 1]^T$. The ray in world coordinates is then obtained as

$$\mathbf{d}_w = \mathbf{R}_w \mathbf{d}, \quad \mathbf{o}_w = \mathbf{t}_w \quad (2)$$

where $\mathbf{R}_w$ and $\mathbf{t}_w$ are the rotation and translation of the camera in world coordinates. The 3D center estimate is then found by minimizing the sum of squared distances to all five rays via least squares:

$$\mathbf{p}^* = \left(\sum_i (\mathbf{I} - \hat{\mathbf{d}}_i \hat{\mathbf{d}}_i^T) \right)^{-1} \sum_i (\mathbf{I} - \hat{\mathbf{d}}_i \hat{\mathbf{d}}_i^T) \mathbf{o}_i, \quad (3)$$

where $\hat{\mathbf{d}}_i$ is the unit direction of ray i and $\mathbf{o}_i$ is its origin.

C. Data Collection

We first perform offline planning (viewpoint and robot joint position generation), then do online data collection via motion planning.

1) *Offline planning*: We create candidate camera viewpoints distributed on a hemisphere centered on the object, with its axis is along the $-X$ axis of the robot base frame. We restrict coverage to a hemisphere rather than a full sphere because the arm’s joint limits prevent the camera from reaching the far side of the sample, and because the stand itself physically blocks a 360° view. As a result, the backside of the sample is never observed. Viewpoints are selected from a Fibonacci lattice [24] for near-uniform angular coverage.

Camera orientations are assigned by a look-at construction: the camera’s optical axis (Z -axis) follows the outward surface normal of the hemisphere, and the camera’s X -axis is the projection of the world $+Z$ onto the hemisphere surface. The viewpoints are then re-ordered via a nearest-neighbor greedy tour on 3D distance to minimize distance traveled for the robot arm between the viewpoints.

The orientation produced by the hemisphere sampling aims the camera’s optical axis ($\mathbf{e}_z$) at the sample. However, the sample should appear at the image center $(W/2, H/2)$, where W and H are the image width and height in pixels,

rather than at the principal point (c_x, c_y) , to ensure that majority of an sample is seen in every captured image frame. We therefore apply a small in-plane correction to each viewpoint orientation. Let $\mathbf{K}$ be the camera intrinsics. The viewing direction in the camera frame corresponding to the image-center pixel is

$$\mathbf{d} = \frac{\mathbf{K}^{-1} \tilde{\mathbf{p}}_c}{\|\mathbf{K}^{-1} \tilde{\mathbf{p}}_c\|}, \quad \tilde{\mathbf{p}}_c = \begin{bmatrix} W/2 \\ H/2 \\ 1 \end{bmatrix}. \quad (4)$$

We then construct the rotation $\mathbf{R}_{\text{off}}$ that aligns this direction with the optical axis $\mathbf{e}_z = [0, 0, 1]^T$. Its angle and axis are

$$\theta = \arccos(\mathbf{d} \cdot \mathbf{e}_z), \quad \hat{\mathbf{a}} = \frac{\mathbf{d} \times \mathbf{e}_z}{\|\mathbf{d} \times \mathbf{e}_z\|}. \quad (5)$$

Then Rodrigues’ formula yields

$$\mathbf{R}_{\text{off}} = \mathbf{I} + \sin \theta [\hat{\mathbf{a}}]_{\times} + (1 - \cos \theta) [\hat{\mathbf{a}}]_{\times}^2, \quad (6)$$

where $[\hat{\mathbf{a}}]_{\times}$ is the skew-symmetric matrix of $\hat{\mathbf{a}}$. Finally, each viewpoint orientation $\mathbf{R}_{\text{wp}}$ is corrected in the camera frame:

$$\mathbf{R}_{\text{new}} = \mathbf{R}_{\text{wp}} \mathbf{R}_{\text{off}}. \quad (7)$$

The camera positions on the hemisphere are left unchanged; only the orientation is tilted so that the sample is centered in the image frame.

Using $\mathbf{R}_{\text{off}}$ and the eye-in-hand transform from the calibration step allows us to find the desired end-effector pose for each viewpoint. These end-effector poses are converted to robot joint values via the stereographic Shoulder-Elbow-Wrist (SEW) inverse kinematics (IK) method [25], which allows us to enumerate all IK solutions of the 7-DOF KUKA robot arm analytically. We set the SEW angle to 135° , which yields eight kinematically feasible robot joint solutions per viewpoint. We then filter each joint solution to avoid collisions with the table and sample, and joint limits, and remove any viewpoints that have no feasible IK solution. Small differences in the localized sample location leave us with either 134 or 135 reachable viewpoints.

2) *Online data collection*: The robot traverses between each viewpoint via the Rapidly-exploring Random Tree* (RRT*) algorithm [26] through a path that avoids any collisions, implemented in Drake [23]. At each viewpoint, the robot takes 30 images for the focal stack, jogging the robot a total of 40 mm closer to the sample along the camera’s optical axis. The robot pauses 0.5 s before taking an image to avoid motion blur. Additionally, the robot stops for 2 s when moving between viewpoints to ensure the exposure and white balance stabilize before taking pictures.

D. Pre-Processing

For accurate 3D reconstruction, we opted to keep the 3DGS method standard and instead clean up the image data. To do so, we create an automated focus stacking pipeline to select which images to stack together, and a strategy for autonomously masking the focus-stacked image.

1) *Focus stacking*: Not all images taken in the focal stack have high sharpness content, and attempting to stack the entire focal stack could result in distorted images. Additionally, selecting too large a range of images for the focal stack may result in artifacts due to focus breathing. Therefore, for every focal stack, it is necessary to select only a subset of the sharpest images.

Due to the poor image quality of the camera and uncontrollable camera image parameters, we cannot use the typical thresholding techniques used in works such as Orbitoscope [6]. We found that the Tenengrad sharpness score of a sequence of 30 images tends to be roughly Gaussian, and an additional pre-processing step allowed for robust window selection within our focal stack.

For a given viewpoint and its sequence of 30 images, we first apply 2D spatial median filtering (5×5 kernel) to each image, since salt-and-pepper noise from the low-cost sensor creates spurious high-gradient pixels that spike the Tenengrad score independent of actual focus. A scalar Tenengrad sharpness score T_i is then computed for each image $i \in \{1, \dots, 30\}$ as the sum of squared horizontal and vertical Sobel gradient responses $G_x(x, y)$ and $G_y(x, y)$ over all pixel coordinates (x, y) in the image domain Ω :

$$T_i = \sum_{(x,y) \in \Omega} (G_x(x, y)^2 + G_y(x, y)^2) \quad (8)$$

While spatial filtering addresses in-frame noise, the resulting sequence of sharpness scores $\{T_1, T_2, \dots, T_{30}\}$ across the focal stack can still exhibit minor fluctuations and outliers. To smooth this score distribution across the image sequence, we apply a subsequent 1D median filter (kernel size 3) directly to the sequence of scalar scores.

To model the sharpness profile across the focal stack, we fit a four-parameter Gaussian function $f(x)$ over the image indices $x \in [1, 30]$:

$$f(x; a, \mu, \sigma, c) = a \exp\left(-\frac{1}{2} \left(\frac{x - \mu}{\sigma}\right)^2\right) + c \quad (9)$$

where a represents the peak amplitude, μ the focal peak position, σ the depth-of-field spread, and c a baseline offset. The model parameters are estimated from the filtered sharpness scores using bounded nonlinear least-squares optimized via the Trust Region Reflective algorithm [27] (`scipy.optimize.curve_fit` [28]).

Images within one standard deviation ($\mu \pm \sigma$) of the estimated focal peak are selected for fusion. Taking the middle image of this subset as the reference frame, we apply complex wavelet-based Extended Depth-of-Field (EDOF) [29] to generate the final focus-stacked image.

2) *Masking*: Isolating the sample from the out-of-focus background is crucial to mitigate floater artifacts during 3DGS optimization. To achieve this without manual annotation, we first construct a mask using classical edge cues at a single, well-conditioned viewpoint, then propagate it to every other viewpoint with SAM 2 [14], whose video mode

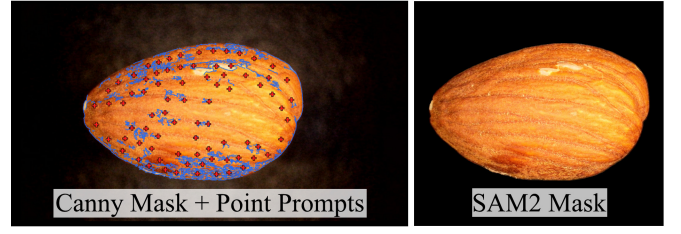


Fig. 5. Example point-prompt generation for SAM 2 segmentation. (Left) Canny edge detection mask in blue, with sampled point prompts in red are used to query SAM 2. (Right) Resulting segmentation mask produced by SAM 2.

conditions on this seed mask and generalizes far better than edge detection once the viewpoint changes.

At the first focus-stacked image, the sample is well-centered and the mounting pole is occluded, making classical edges reliable: we extract edges via a 5×5 Gaussian blur followed by Canny edge detection [30] and densely sample $n = 80$ edge points as foreground prompts for SAM 2 in image-prediction mode. The resulting mask is slightly dilated and flood-filled, then used to seed SAM 2 in video mode (Fig. 5). This handoff is necessary because, at other viewpoints, the pole becomes visible and lighting shifts, causing edge-based segmentations to fail.

E. Reconstruction

Our reconstruction consists of COLMAP and 3DGS processes, both adjusted for better accuracy.

1) *Bundle Adjustment*: We use COLMAP [31, 32] to refine the camera intrinsics and extrinsics, and to generate a sparse point cloud for initializing the 3DGS optimization. First, SIFT [33] features are extracted and exhaustively matched across all views. COLMAP’s incremental mapper is then seeded with our initial camera intrinsics and the robot’s reported end-effector poses. We let COLMAP refine these further, accounting for errors in intrinsics, as well as small, unmodeled pose offsets from the custom microscope mount.

2) *3D Gaussian Splatting*: We use the `gsplat` library’s [34] implementation of 3DGS [5]. To suppress floater artifacts while preserving fine structural details, we adjust the default densification hyperparameters, increasing the opacity pruning threshold to $\tau_\alpha = 0.10$ and the 2D gradient growth threshold to $\tau_\nabla = 4 \times 10^{-4}$.

Additionally, we introduce a background alpha regularization term that penalizes rendered opacity in pixels outside the generated sample masks:

$$\mathcal{L}_{bg} = \frac{1}{|\mathcal{B}|} \sum_{(u,v) \in \mathcal{B}} \hat{\alpha}_{uv}, \quad (10)$$

where $\mathcal{B}$ represents the set of background pixels defined by our segmentation masks, and $\hat{\alpha}_{uv}$ is the rendered alpha value at pixel location (u, v) . This background drives the opacity of background Gaussians toward zero, ensuring most floaters are removed during the pruning phase. We set the background loss scaling factor $\lambda_{bg} = 0.2$.

Method	Almond			Blueberry			Bolt			Circuit			Flower			Pinecone			Rock			Seashell		
Vanilla 3DGS	27.58	0.65	0.20	25.24	0.75	0.29	22.70	0.59	0.17	18.18	0.48	0.16	23.68	0.74	0.18	23.47	0.59	0.25	30.63	0.77	0.22	28.03	0.86	0.16
CoCoGaussian	27.31	0.64	0.19	24.57	0.71	0.30	18.72	0.39	0.20	17.97	0.48	0.15	20.02	0.62	0.19	18.60	0.51	0.30	29.51	0.75	0.22	27.01	0.83	0.15
Deblurring 3DGS	26.32	0.62	0.21	23.30	0.66	0.29	18.83	0.41	0.21	17.53	0.46	0.17	22.78	0.74	0.18	19.25	0.51	0.29	29.93	0.77	0.22	24.64	0.79	0.17
BAGS	29.48	0.70	0.17	27.21	0.79	0.26	22.78	0.59	0.16	19.77	0.56	0.14	26.39	0.79	0.16	26.29	0.68	0.20	32.70	0.80	0.19	30.64	0.89	0.14
Ours	33.49	0.96	0.06	37.45	0.98	0.02	36.06	0.97	0.04	39.53	0.99	0.01	32.17	0.95	0.06	41.68	0.99	0.01	45.39	0.99	0.01	45.13	1.00	0.01

Table 1. Quantitative comparisons across all samples in the synthetic dataset. Top three per metric are colored red, orange, and yellow respectively.

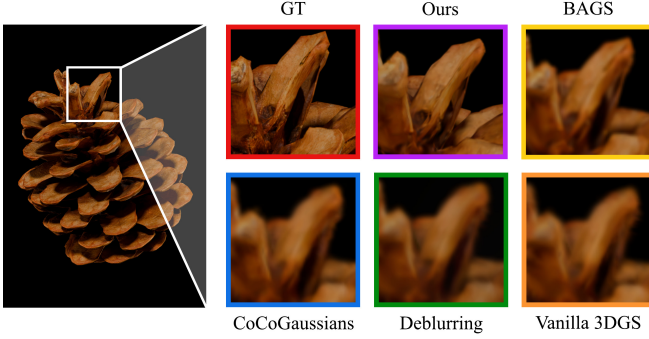


Fig. 6. Qualitative comparison on a pinecone from our synthetic dataset.

Combined with the standard 3DGS photometric objective, the complete loss minimized during training is:

$$\mathcal{L} = (1 - \lambda_{\text{ssim}}) \mathcal{L}_1 + \lambda_{\text{ssim}} \mathcal{L}_{\text{SSIM}} + \lambda_{\text{bg}} \mathcal{L}_{\text{bg}}, \quad (11)$$

where $\mathcal{L}_1$ is the per-pixel photometric loss, and $\mathcal{L}_{\text{SSIM}}$ is the D-SSIM structural similarity loss between the rendered and ground-truth images. We leave the SSIM loss scaling factor $\lambda_{\text{ssim}} = 0.2$ unchanged at its default value from [5]. Lastly, following [5], every eighth image is held out for evaluation during training.

IV. EXPERIMENTS

We now experimentally validate the MicroSplat pipeline. First, we ablate the object-centering step to see how well it can localize and keep an object centered during data collection. Second, we test various SfM initialization methods to see which version produces the best results.

We also benchmark against vanilla 3DGS [5] (trained on images that neither focus-stacked or masked) and three state-of-the-art deblurring approaches in 3DGS: BAGS [20], CoCoGaussian [21], and Deblurring 3DGS [19].

A. Runtime

On average, the full MicroSplat pipeline takes 134 minutes per object on a single NVIDIA L40S: 2 minutes for object-centering, 39 minutes for data collection, 33 minutes for pre-processing (focus-stacking and masking), and 60 minutes for the reconstruction (COLMAP and 3DGS).

B. 3DGS Training Parameters

We follow the original 3DGS [5] training configuration, optimizing for 30,000 iterations from the COLMAP sparse point cloud with input images downsampled by a factor of 4 from the microscope’s 1920 × 1080 resolution, and use identical settings for both the real and synthetic datasets.

We deviate from the defaults in two places, both chosen to help with suppressing floater artifacts: the opacity pruning

threshold τ_α is doubled from 0.05 to 0.10, and the 2D gradient growth threshold τ_∇ also doubled from 2×10^{-4} to 4×10^{-4} .

C. Datasets

1) *Real-World Dataset*: To qualitatively evaluate the system, we selected eight samples (almond, circuit, blueberry, bolt, rock, pinecone, hydrangea, and seashell) spanning surface appearance (matte to slightly specular), brightness, geometric complexity, texture frequency, and depth extent. Each sample’s maximum width is 15-27 mm.

2) *Synthetic Dataset*: We quantitatively evaluate focus stacking in simulation because no reliable ground truth exists for the real captures: every real image is degraded by the shallow DoF the pipeline is designed to correct, so there is no in-focus reference to score against.

We generate a synthetic Blender dataset of 8 samples, modeled after the same objects as the real-world set. Camera poses are inherited from the real-world almond COLMAP reconstruction, which recovers a single refined pose per viewpoint — that of the focus-stacked image. Since each stack sweeps 40 mm over 30 images, we recover the remaining 29 poses by applying the corresponding axial offset from the selected stack index. Each synthetic model is then scaled and positioned to sit within these reconstructed scan points.

Since the microscope’s manufacturer does not specify its aperture or focus distance, we recover it from the 42 images already captured for intrinsic calibration (Section III), fitting a thin-lens defocus model to per-corner blur and depth (from PnP) on the well-conditioned views, yielding a median working f-number of $N = 6.2$ and focus distance of 104 mm.

Poses	Intrinsics	PSNR $\uparrow$		SSIM $\uparrow$		LPIPS $\downarrow$	
		Mean	Std.	Mean	Std.	Mean	Std.
No init.	No init.	19.42	5.04	0.672	0.145	0.389	0.186
Init.	No init.	21.65	2.19	0.701	0.076	0.365	0.128
No init.	Init.	20.81	4.42	0.697	0.157	0.363	0.201
Fixed	No init.	16.39	2.19	0.593	0.104	0.554	0.130
Fixed	Fixed	17.35	1.53	0.592	0.092	0.507	0.116
Fixed	Init.	16.58	2.15	0.607	0.093	0.518	0.128
Init.	Init.	26.37	0.28	0.911	0.002	0.123	0.003

Table 2. SfM initialization ablation (averaged across real objects). For robot poses and calibrated intrinsics, No init. denotes bundle adjustment by COLMAP without a prior, Init. denotes initialization from the prior followed by refinement, and Fixed denotes freezing to the prior. The bottom row is our configuration. Top three per metric are colored red, orange, and yellow, respectively.

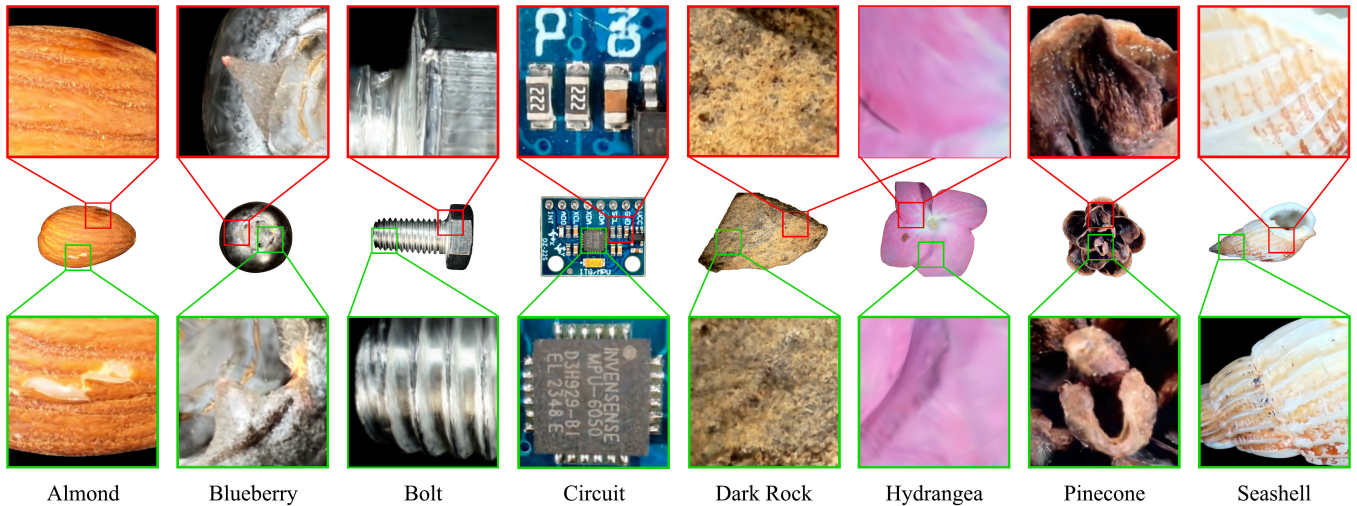


Fig. 7. Qualitative real-world reconstruction results.

D. Ablation Study

1) *Object-Centering Experiment*: We evaluate the object-centering algorithm on a spherical mocap marker, whose center is well-defined from any viewing angle. After running the centering and data-collection stages, we report the pixel distance between the sample’s centroid and the true image center across all scan poses. For each viewpoint, we focus-stack the images as in the main pipeline, then manually draw a bounding box on the focus-stacked image and take its center.

The results show an average pixel error of 98 px, which is consistent with the combined effect of the pipeline’s known error sources. The 1.07 mm hand-eye calibration residual and the 1.0 mm manual jogging increment suggest the centering accuracy is limited by the mount’s imperfect rigidity and discrete jogging precision rather than the ray-intersection procedure itself.

2) *SfM Initialization Priors*: To assess the robot’s reported poses and our calibrated intrinsics as SfM priors, we compare configurations in which each prior is unused, used to initialize bundle adjustment, or fixed, evaluated on the real-world dataset (Table 2).

Fixing the robot poses performs worst (PSNR <18 dB) regardless of how intrinsics are handled. Although the arm itself is precise, the mount’s imperfect rigidity leave the derived camera poses too imprecise for high-fidelity rendering without refinement. Initializing with either prior alone improves over using neither (+2.23 dB for poses, +1.39 dB for intrinsics), but initializing with both yields the largest gain (+6.95 dB) and reduces the standard deviation across objects from 5.04 dB to 0.28 dB, indicating the priors are complementary.

E. Comparisons

To compare against baseline methods, our method is initialized with the robot’s reported poses and calibrated intrinsics and lets COLMAP refine them, mirroring how the pipeline runs on real data. Each baseline uses the same parameters from its publication, with minor adjustments to

accommodate our sample sizes. For these methods, SfM on the defocused inputs produced obvious pose errors, so we instead supply known intrinsics and frozen ground-truth poses. This is an advantage our method does not receive. We run the blur-aware baselines without masks, as published, since they model depth-dependent defocus; to give them realistic defocus to model, our synthetic scenes include a background modeled after our real-world workspace.

Our method consistently outperforms the blur-aware methods (Table 1, Figure 6), as they must infer sharp detail, while the arm’s controlled focal sweep ensures it is measured directly. The gap is especially large because many of our frames are fully out of focus. Deblurring 3DGS and CoCo-Gaussian realize blur through the Gaussians themselves, so any blur they cannot represent is absorbed into geometry, leaving them below even vanilla 3DGS. BAGS applies its blur as a 2D convolution on the final render, leaving geometry untouched, and performs best among the baselines.

F. Real-World Results

The synthetic dataset controls lighting, exposure, and white balance; the real captures have none of these guarantees, so these results demonstrate the pipeline’s ability to extrapolate to our low-cost microscope. As shown in Figure 7, many of the reconstructions are difficult to distinguish from photographs. Intricate detail survives: the peeling skin on the almond, the flap on the blueberry, and the even the text on the circuit board, whose characters are roughly 0.4 mm wide. The hydrangea is the weakest result. Its petals are thin, which the volumetric 3D Gaussian representation is known to handle poorly [35], and its uniform coloring yields sparse SfM initialization.

V. CONCLUSION

We presented MicroSplat, a robotic pipeline that turns a low-cost consumer microscope on a KUKA iiwa14 R820 into an autonomous scanner for centimeter-scale objects. After calibration and a brief object-centering step, it runs unattended from capture to a finished 3DGS model with

sub-millimeter detail. We also release the real and synthetic datasets used in this evaluation.

The system is still limited in many ways; it is not fully autonomous, and our reconstructions do not provide full 360° coverage. Looking further ahead, we believe the solutions we developed for lighting inconsistency, pose inaccuracy, and non-uniform backgrounds will be useful for a future robotic system capable of scanning and reconstructing objects in the wild without disturbing them.

REFERENCES

- [1] Z. Fu, B. Qiu, J. Shen, and N. Chen, “3d reconstruction of pcb based on icp point cloud optimization,” in *2023 2nd International Conference on Big Data, Information and Computer Network (BDICN)*, 2023, pp. 317–321.
- [2] F. Ulger, S. E. Yuksel, A. Yilmaz, and D. Gokcen, “Solder joint inspection on printed circuit boards: A survey and a dataset,” *IEEE Transactions on Instrumentation and Measurement*, vol. 72, 2023.
- [3] L. Macdonald, V. Moitinho, and M. Hess, “Three-dimensional reconstruction of roman coins from photometric image sets,” *Journal of Electronic Imaging*, vol. 26, p. 011017, Feb. 2017.
- [4] D. Thanh-Nghi and C. Nguyen, “A low-cost digital 3d insect scanner,” *Information Processing in Agriculture*, vol. 11, Mar. 2023.
- [5] B. Kerbl, G. Kopanas, T. Leimkühler, and G. Drettakis, *3d gaussian splatting for real-time radiance field rendering*, 2023.
- [6] T. A. Yliniemi and U. Kõljalg, “The Orbitoscope, a six-axis macro-imaging robot for photogrammetric 3D-digitization of insects and other small specimens,” *ZooKeys*, vol. 1277, p. 137, 2026.
- [7] D. Bittel, *Macro splats 2025*, <https://danybittel.ch/macro.html>, Accessed: 2025, 2025.
- [8] B. Mildenhall, P. P. Srinivasan, M. Tancik, J. T. Barron, R. Ramamoorthi, and R. Ng, “Nerf: Representing scenes as neural radiance fields for view synthesis,” *CoRR*, vol. abs/2003.08934, 2020.
- [9] L. Downs, A. Francis, N. Koenig, B. Kinman, R. Hickman, K. Reymann, T. B. McHugh, and V. Vanhoucke, *Google scanned objects: A high-quality dataset of 3d scanned household items*, 2022.
- [10] T. Qiu, Z. Ma, K. El-Refai, H. Shah, C. M. Kim, J. Kerr, and K. Goldberg, *Omni-scan: Creating visually-accurate digital twin object models using a bimanual robot with handover and gaussian splat merging*, 2025.
- [11] M. Strong, B. Lei, A. Swann, W. Jiang, K. Daniilidis, and M. K. III, *Next best sense: Guiding vision and touch with fisherrf for 3d gaussian splatting*, 2025.
- [12] W. Jiang, B. Lei, and K. Daniilidis, *Fisherrf: Active view selection and uncertainty quantification for radiance fields using fisher information*, 2024.
- [13] X. Pan, Z. Lai, S. Song, and G. Huang, *Activenerf: Learning where to see with uncertainty estimation*, 2022.
- [14] N. Ravi, V. Gabeur, Y.-T. Hu, R. Hu, C. Ryali, T. Ma, H. Khedr, R. Rädle, C. Rolland, L. Gustafson, E. Mintun, J. Pan, K. V. Alwala, N. Carion, C.-Y. Wu, R. Girshick, P. Dollár, and C. Feichtenhofer, *Sam 2: Segment anything in images and videos*, 2024.
- [15] A. Gallo, M. Muzzupappa, and F. Bruno, “3d reconstruction of small sized objects from a sequence of multi-focused images,” *Journal of Cultural Heritage*, vol. 15, pp. 173–182, Mar. 2014.
- [16] K. Kikuchi, P. Cabrera, J. Victoriano, J. Rofes, and G. Tapang, “Three dimensional models from microscopy images using gaussian splatting,” Jul. 2024.
- [17] W. Wang and F. Chang, “A multi-focus image fusion method based on laplacian pyramid,” *Journal of Computers*, vol. 6, pp. 2559–2566, Dec. 2011.
- [18] P.-E. Edeline, M. Leclercq, J. le luyer, A. Droit, and S. Chabrier, “3d microscopic reconstruction of pearls using combined optical microscopy and photogrammetry,” *Scientific Reports*, vol. 14, Jun. 2024.
- [19] B. Lee, H. Lee, X. Sun, U. Ali, and E. Park, *Deblurring 3d gaussian splatting*, 2024.
- [20] C. Peng, Y. Tang, Y. Zhou, N. Wang, X. Liu, D. Li, and R. Chellappa, “BAGS: Blur agnostic Gaussian splatting through multi-scale kernel modeling,” in *European Conference on Computer Vision (ECCV)*, 2024.
- [21] J. Lee, S. Cho, T. Kim, H.-D. Jang, M. Lee, G. Cha, D. Wee, D. Lee, and S. Lee, *Cocogaussian: Leveraging circle of confusion for gaussian splatting from defocused images*, 2025.
- [22] K. Daniilidis and E. Bayro-Corrochano, “The dual quaternion approach to hand-eye calibration,” in *Proceedings of 13th International Conference on Pattern Recognition*, vol. 1, 1996, 318–322 vol.1.
- [23] R. Tedrake and the Drake Development Team, *Drake: Model-based design and verification for robotics*, 2019.
- [24] Á. González, “Measurement of areas on a sphere using fibonacci and latitude–longitude lattices,” *Mathematical Geosciences*, vol. 42, no. 1, pp. 49–64, Nov. 2009.
- [25] A. J. Elias and J. T. Wen, *Redundancy parameterization and inverse kinematics of 7-dof revolute manipulators*, 2024.
- [26] S. Karaman and E. Frazzoli, *Sampling-based algorithms for optimal motion planning*, 2011.
- [27] M. Branch, T. Coleman, and Y. li, “A subspace, interior, and conjugate gradient method for large-scale bound-constrained minimization problems,” *SIAM Journal on Scientific Computing*, vol. 21, pp. 1–23, Jul. 2006.
- [28] P. Virtanen, R. Gommers, T. E. Oliphant, M. Haberland, T. Reddy, D. Cournapeau, E. Burovski, P. Peterson, W. Weckesser, J. Bright, S. J. van der Walt, M. Brett, J. Wilson, K. J. Millman, N. Mayorov, A. R. J. Nelson, E. Jones, R. Kern, E. Larson, C. J. Carey, Í. Polat, Y. Feng, E. W. Moore, J. VanderPlas, D. Laxalde, J. Perktold, R. Cimrman, I. Henriksen, E. A. Quintero, C. R. Harris, A. M. Archibald, A. H. Ribeiro, F. Pedregosa, P. van Mulbregt, and SciPy 1.0 Contributors, “SciPy 1.0: Fundamental algorithms for scientific computing in Python,” *Nature Methods*, vol. 17, pp. 261–272, 2020.
- [29] B. Forster-Heinlein, D. Ville, J. Berent, D. Sage, and M. Unser, “Complex wavelets for extended depth-of-field: A new method for the fusion of multichannel microscopy images,” *Microscopy research and technique*, vol. 65, pp. 33–42, Sep. 2004.
- [30] J. Canny, “A computational approach to edge detection,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. PAMI-8, pp. 679–698, Dec. 1986.
- [31] J. L. Schönberger and J.-M. Frahm, “Structure-from-motion revisited,” in *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 4104–4113.
- [32] J. L. Schönberger, E. Zheng, J.-M. Frahm, and M. Pollefeys, “Pixelwise view selection for unstructured multi-view stereo,” in *Computer Vision – ECCV 2016*, B. Leibe, J. Matas, N. Sebe, and M. Welling, Eds., Cham: Springer International Publishing, 2016, pp. 501–518.
- [33] D. Lowe, “Distinctive image features from scale-invariant keypoints,” *International Journal of Computer Vision*, vol. 60, pp. 91–110, Nov. 2004.
- [34] V. Ye, R. Li, J. Kerr, M. Turkulainen, B. Yi, Z. Pan, O. Seiskari, J. Ye, J. Hu, M. Tancik, and A. Kanazawa, *Gsplat: An open-source library for gaussian splatting*, 2024.
- [35] B. Huang, Z. Yu, A. Chen, A. Geiger, and S. Gao, “2d gaussian splatting for geometrically accurate radiance fields,” in *Special Interest Group on Computer Graphics and Interactive Techniques Conference Conference Papers*, ser. SIGGRAPH ’24, ACM, 2024, pp. 1–11.